

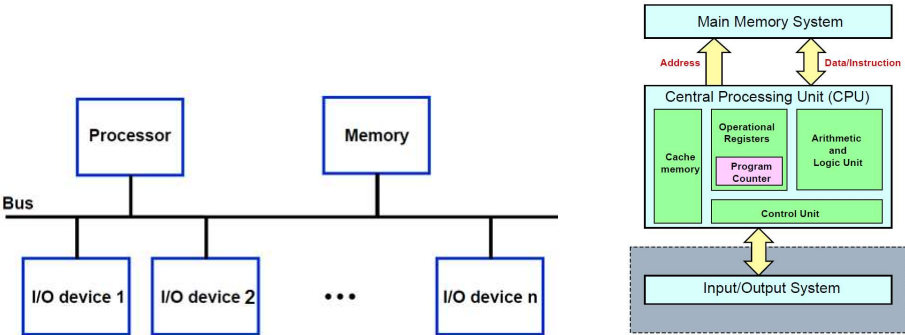
COMPUTER ARCHITECTURE



Hamid Sarbazi-Azad
Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

I/O Organization

- In a computer system (with a common bus):
- all devices connected to the bus are able to exchange information
 - the bus consists of separate Address, Data, and Control lines,
 - each I/O device has a unique set of addresses.



I/O Mapping

➤ Memory mapped I/O

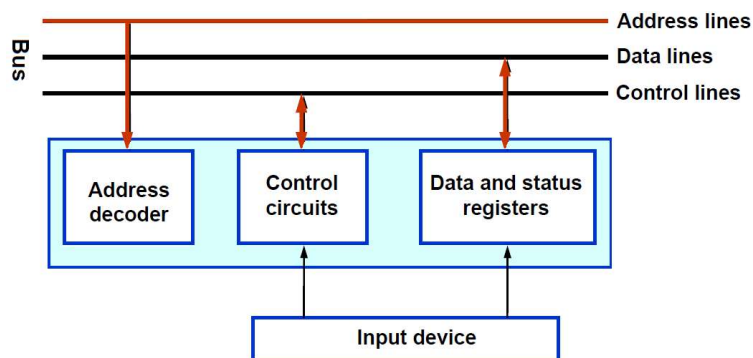
- Devices and memory share a common address space
- I/O device looks just like a memory location and I/O operation is like memory read/write operation
- No special commands/instructions for I/O (thanks to CPU's rich ISA, a large selection of memory access commands are available 😊)

➤ Isolated space I/O

- Separate address spaces for I/O and memory
- Need separate I/O and memory select lines on CPU chip
- Special command/instructions for I/O (CPU's ISA have a limited set of instructions for I/O, e.g. IN and OUT instructions)

I/O Interface for an input/output device

The address decoder, the data and status registers, and control logic to coordinate I/O transfers form the device's interface circuit.



(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

5

I/O Techniques

- Programmed I/O: CPU is involved in checking if I/O device is ready for data movement. If so, CPU transfers the data between the main memory and I/O device.
- Interrupt driven (or Interrupted) I/O: CPU does its normal job. When I/O device has a data to transfer, it interrupts the CPU and in the Interrupt Service Routine, CPU transfers the data between main memory and I/O device.
- Direct Memory Access (DMA): CPU is not involved in data communication with I/O device. The DMA unit takes the role and transfers the data between I/O device and main memory.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

6

I/O Techniques

Consider a system with one input and one output device connected to the bus via a device interface circuit.

The diagram illustrates a device interface circuit connected to a system bus. The bus consists of three lines: Address lines (red), Data lines (black), and Control lines (black). The device interface circuit is divided into three main sections: Address decoder, Control circuits, and Data/status registers. The Address decoder is connected to the Address lines. The Control circuits are connected to the Control lines. The Data/status registers are connected to the Data lines. The Data/status registers include fields for DATAIN, DATAOUT, STATUS (with bits obe and ibf), and CONTROL (with bits 7-0). An Input device is connected to the Control circuits and the STATUS field. An Output device is connected to the Control circuits and the DATAOUT field.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

7

I/O Techniques

Programmed I/O scheme to read one byte from input device can be implemented as follows:

Wait: INAL, IO_device_status
TESTAL,1
JZWait

INAL, IO_device_datain
CALLProcess_data_in_AL
....

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

8

I/O Techniques

Programmed I/O scheme to write one byte to output device can be implemented as follows:

Wait: INAL, IO_device_status
TESTAL,16
JZWait

CALLProduce_data_and_put_it_into_AL
OUTIO_device_dataout,AL
...

4

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

9

I/O Techniques

In programmed I/O, CPU repeatedly checks the status flag of the IO device (polls the IO device) until it gets ready for data transfer.

The diagram illustrates the programmed I/O technique. It shows a CPU connected to a Bus (Address lines, Data lines, Control lines). The CPU's internal components include an Address decoder, Control circuits, and Data/status registers. The Data/status registers are divided into DATAIN, DATAOUT, STATUS, and CONTROL. The STATUS register has a bit labeled 'INTR' (interrupt request). An Input device and an Output device are connected to the Control circuits. The Control circuits send an 'intr' signal to the CPU's Control circuits. The CPU repeatedly checks the STATUS register for the 'INTR' bit to determine when the IO device is ready for data transfer.

Another way that does not involve CPU for status checking of IO device is called **interrupted I/O** technique.

In interrupted I/O, synchronization between CPU and IO device is achieved by a special signal (interrupt request signal) generated by IO device to show that IO device is ready for data transfer.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

I/O Techniques

When an interrupt occurs, CPU leaves its normal work and execute an interrupt service routine.

Inside the interrupt service routine, the data is either:

- read from Datain register, or
- written into Dataout register

of IO Device Control Circuit.

At the end of Interrupt Service Routine, the control is returned to the program which was interrupted.

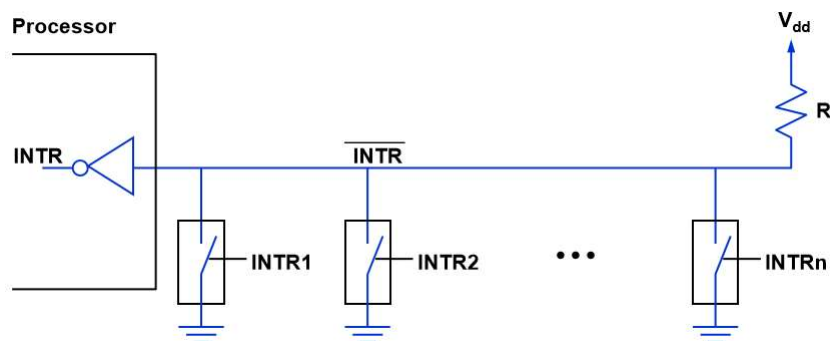
The diagram illustrates the interrupted I/O technique. It shows a CPU connected to a Bus (Address lines, Data lines, Control lines). The CPU's internal components include an Address decoder, Control circuits, and Data/status registers. The Data/status registers are divided into DATAIN, DATAOUT, STATUS, and CONTROL. The STATUS register has a bit labeled 'INTR' (interrupt request). An Input device and an Output device are connected to the Control circuits. The Control circuits send an 'intr' signal to the CPU's Control circuits. The CPU repeatedly checks the STATUS register for the 'INTR' bit to determine when the IO device is ready for data transfer.

The diagram also shows a flowchart of the interrupt service routine. The Main Program is interrupted at point 'i' and jumps to the Interrupt Service Subroutine. The Interrupt Service Subroutine executes and then returns control to the Main Program at point 'i+1'.

I/O Techniques

Interrupt Hardware:

In order to get interrupt request signals from different IO devices, one can use a simple open-drain bus to implement a common interrupt request line.



I/O Techniques

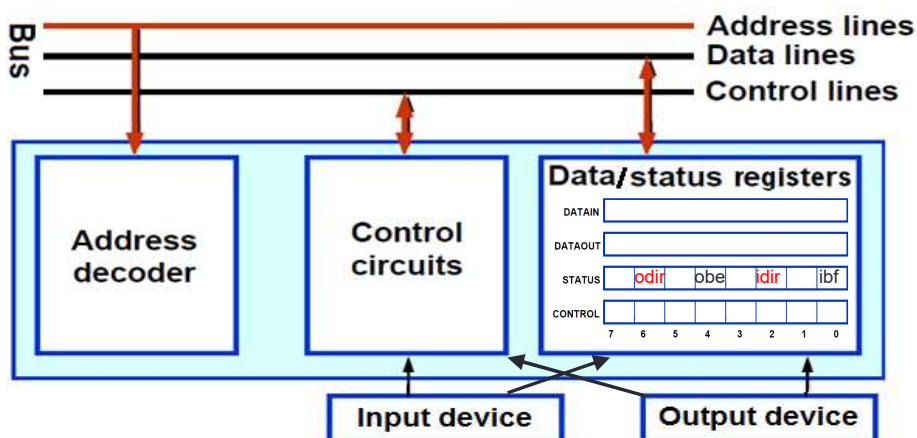
Handling multiple devices raises a number of questions:

- How CPU recognizes the interrupting device?
- Given that different devices require different interrupt service routines, how can CPU obtain the starting address of appropriate routine?
- Should a device be allowed to interrupt CPU while another interrupt is being serviced?
- How two or more simultaneous interrupt requests are handled?

I/O Techniques

Identify the interrupting device:

To check if a device has generated an interrupt, one can read the status register. There is a bit that shows if the device has an outstanding interrupt request.



I/O Techniques

Identify the interrupting device:

The simplest way to identify the interrupting device is to poll all IO devices connected to INTR line. → CPUs time is spent to interrogate all devices ☹

A device requesting an interrupt may identify itself directly to CPU. CPU can then immediately start executing the corresponding interrupt service routine. ☺

→ Vectored Interrupt

When multiple devices can interrupt CPU, what happens if two or more devices generate interrupt request at the same time?

→ Priority can solve the problem ☺

An interrupt from a high-priority device should be accepted while a lower priority device's interrupt is being handled.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

15

I/O Techniques

Implementation of interrupt priority:

Interrupt requests can be examined in parallel and acknowledged.

The diagram illustrates a parallel interrupt controller. On the left, a light blue box labeled 'Processor' contains a vertical 'Interrupt Controller'. To the right, several devices are shown: 'Device 1', 'Device 2', an ellipsis '...', and 'Device p'. Each device has an input line from the processor labeled $\overline{\text{INTR}}1$, $\overline{\text{INTR}}2$, ..., $\overline{\text{INTR}}p$ and an output line to the processor labeled $\text{INTA}1$, $\text{INTA}2$, ..., $\text{INTA}p$. The interrupt controller is connected to all these lines, allowing it to examine multiple requests in parallel and send individual acknowledgments back to each device.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

16

I/O Techniques

Implementation of interrupt priority:

A 8-input Parallel Interrupt Controller with fixed priority scheme.

This diagram shows a more detailed implementation of a parallel interrupt controller with a fixed priority scheme. On the left, eight input lines are labeled $\text{INTR } 7$ through $\text{INTR } 0$. Each input line passes through an inverter and then into an 8-input AND gate. The output of each AND gate is an acknowledgment line labeled $\text{ACK } 7$ through $\text{ACK } 0$. The logic ensures that only the highest-priority active interrupt is acknowledged. On the right, a smaller version of the diagram from slide 15 is shown, with the 'Interrupt Controller' block connected to the processor and the various devices (Device 1, Device 2, ..., Device p) via their respective $\overline{\text{INTR}}$ and INTA lines.

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

17

I/O Techniques

Implementation of interrupt priority:

Interrupt requests can be examined in a serial manner called Daisy Chain scheme. → Interrupt Controller Unit is not needed 😊

When an interrupt request is activated, CPU acknowledge its acceptance by generating INTA signal. This signal goes through different devices starting from the one with highest priority. If it has an outstanding interrupt request, it give its vector in Interrupt Acknowledge cycle and inactivates the passed signal to the next device. If it is not the interrupting device, it passes the INTA signal to the next device, and so on.

```
graph LR
    Processor[Processor]
    Device1[Device 1]
    Device2[Device 2]
    Dots[...] 
    Devicen[Device n]
    Processor -- INTR --> Device1
    Processor -- INTR --> Device2
    Processor -- INTR --> Devicen
    Processor -- INTA --> Device1
    Device1 --> Device2
    Device2 --> Dots
    Dots --> Devicen
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

18

I/O Techniques

Implementation of interrupt priority:

A mix of the above schemes can also be implemented for a large number of interrupting devices.

```
graph LR
    subgraph IC [Interrupt Controller]
        INTR1_in[INTR1]
        INTRp_in[INTRp]
        INTA1_out[INTA1]
        INTAp_out[INTAp]
    end
    Device1_1[Device]
    Device2_1[Device]
    Dots_1[...]
    Devicen_1[Device]
    Device1_p[Device]
    Device2_p[Device]
    Dots_p[...]
    Devicen_p[Device]
    INTR1_in --> Device1_1
    Device1_1 --> Device2_1
    Device2_1 --> Dots_1
    Dots_1 --> Devicen_1
    INTRp_in --> Device1_p
    Device1_p --> Device2_p
    Device2_p --> Dots_p
    Dots_p --> Devicen_p
    INTA1_out --> Device1_1
    INTAp_out --> Device1_p
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

19

I/O Techniques

Direct Memory Access (DMA):

To transfer large blocks of data at high speed, a special unit can control the data transfers between main memory and IO devices without involving CPU in the process. It is called **Direct Memory Access (DMA)** Unit.

Since it transfers blocks of data, it must keep tracking source and destination addresses and the number of transfers done.

```
graph TD
    Processor[Processor] --- Bus[System bus]
    MainMemory[Main memory] --- Bus
    Bus --- DiskController[Disk controller]
    Bus --- DMAController[DMA controller]
    Bus --- Printer[Printer]
    Bus --- Keyboard[Keyboard]
    DiskController --- Disk1[Disk]
    DiskController --- Disk2[Disk]
```

(c) Hamid Sarbazi-Azad

Computer Architecture -- Lecture #8: Input/Output Organization

20

I/O Techniques

Direct Memory Access (DMA):

It has special registers to keep the address of source and destination, the volume of transfer, and the type of devices involved and the type of data transfer.

The diagram illustrates the internal structure of the DMA Unit and its connection to the system bus. The unit is divided into three main sections: an Address decoder, Control circuits, and Address/status registers. The Address decoder is connected to the Address lines of the bus. The Control circuits are connected to the Control lines of the bus. The Address/status registers are connected to both the Address lines and the Data lines of the bus. The registers include fields for source address, destination address, transfer count, status, and control.

Address/status registers			
source address			
destin. address			
transfer count			
STATUS		...	
CONTROL		...	

DMA Unit

I/O Techniques

Direct Memory Access (DMA):

A conflict may arise if CPU and DMA Unit (or even more DMA Units) try to use the common bus.

To resolve the problem, a bus arbiter unit (or a bus controller unit) must be used.

CPU, DMA, coprocessors, ... or any other bus master may take the control of bus and use it and release it. If a bus master needs the bus, it should ask the bus arbiter. Arbiter checks its priority and request history and if decides to grant it to access the bus, the requesting bus master should wait until the current bus master release the bus. It can then get the bus and be the current bus master.

I/O Techniques

Summary of I/O techniques:

- Programmed I/O is **simplest** technique but **wastes CPU time** for both checking the readiness of IO device and actual data transfer between IO device and main memory.
- Interrupted I/O **works better** but still involves CPU and **wastes its time** for actual data transfer between IO device and main memory.
- DMA is **best** especially for large data transfers between IO devices and main memory, between IO devices, and within main memory. It does **not waste** CPU time for data transfer. However, it is **costly**.

END OF SLIDES

QUESTIONS?